

Informe sobre React

--- SECCIÓN 1: INTRODUCCIÓN Y CONCEPTOS CLAVE ---

1. ¿Cuál es la definición general de React.js y cuál es su propósito principal en el desarrollo web?

Es una biblioteca de JavaScript diseñada para construir interfaces de usuario de forma declarativa y basada en componentes[cite: 1]. Su propósito principal es gestionar y renderizar de forma eficiente la capa de la vista (UI) en aplicaciones web o móviles[cite: 1].

2. ¿Qué significa que React sea una biblioteca de naturaleza "declarativa"? ¿En qué se diferencia del enfoque imperativo?

Significa que el desarrollador describe CÓMO debe verse la interfaz de usuario en función del estado actual[cite: 1], en lugar de programar paso a paso las instrucciones manuales para actualizar el DOM (enfoque imperativo)[cite: 1].

3. ¿Cómo define el documento el concepto de "basado en componentes"?

Lo define como la construcción de interfaces a través de piezas de código encapsuladas y reutilizables[cite: 1].

4. ¿Qué característica le permite a React ser "multiplataforma" y qué entornos de ejecución abarca?

La separación entre la lógica de cálculo y la capa de renderizado ("Escribe una vez, pinta donde sea")[cite: 1]. Abarca la web (mediante react-dom) y entornos móviles como iOS y Android (mediante react-native)[cite: 1].

--- SECCIÓN 2: PARADIGMA: LIBRERÍA VS. FRAMEWORK ---

5. Según la comparación con frameworks como Angular o Vue, ¿cuál es la responsabilidad principal de React dentro de una aplicación?

React solo se encarga de actualizar y renderizar la capa de la vista (UI)[cite: 1]. A diferencia de Angular o Vue, no controla toda la arquitectura de la aplicación[cite: 1].

6. ¿Cómo maneja React las áreas de "Enrutamiento" y "Gestión de Estado" en comparación con los frameworks tradicionales?

React no tiene una opinión predeterminada y requiere herramientas externas para el enrutamiento (como react-router)[cite: 1]. En la gestión de estado es flexible, permitiendo integrar Context, Redux o Zustand[cite: 1]. Los frameworks tradicionales suelen incluir estas soluciones de forma nativa[cite: 1].

7. ¿Por qué se afirma que React "no te impone una arquitectura completa"?

Porque te otorga el motor para pintar interfaces pero te da la libertad de elegir las librerías complementarias para el resto de la aplicación (rutas, estados, peticiones, etc.)[cite: 1].

--- SECCIÓN 3: ENTORNO Y HERRAMIENTAS ---

8. ¿Qué requisitos previos del entorno de desarrollo (Node.js, gestores de paquetes, editor) son necesarios para trabajar con React?

Node.js (versión superior a v16) como entorno de ejecución backend[cite: 1]; un gestor de paquetes (npm, yarn o pnpm) para instalar dependencias[cite: 1]; y Visual Studio Code como editor de código[cite: 1].

9. ¿Cuáles son las extensiones clave de editor y navegador recomendadas para la productividad y depuración?

Prettier para el formateo automático de código en el editor[cite: 1], y React Developer Tools para la depuración directamente en el navegador[cite: 1].

10. ¿Qué comandos de la terminal se utilizan para inicializar, instalar dependencias y ejecutar un proyecto con Vite?

- Crear: `npm create vite@latest mi-app-react -- --template react`[cite: 1]
- Navegar: `cd mi-app-react`[cite: 1]
- Instalar: `npm install`[cite: 1]
- Ejecutar: `npm run dev`[cite: 1]

11. ¿Por qué la herramienta "Vite" reemplazó a "create-react-app" en la industria moderna?

Debido a su extrema velocidad, su soporte para recarga en caliente instantánea (HMR) y la generación de empaquetados finales más ligeros[cite: 1].

--- SECCIÓN 4: ANATOMÍA DE UN PROYECTO (ESTRUCTURA DE ARCHIVOS) ---

12. ¿Cuál es la función del directorio "src/" y por qué se considera el laboratorio principal?

Contiene todo el código fuente de la aplicación[cite: 1]. Se considera el laboratorio principal porque es donde el desarrollador pasa el 99% del tiempo escribiendo componentes y lógica[cite: 1].

13. ¿Qué rol cumplen los archivos "index.html" y "main.jsx" al momento de inyectar la aplicación React en el navegador?

"index.html" actúa como el punto de entrada web con el contenedor `<div id="root">`[cite: 1].
"main.jsx" funciona como el script puente que inyecta la aplicación en dicho div utilizando `ReactDOM.createRoot`[cite: 1].

14. ¿Cuál es la utilidad del archivo "package.json" y la carpeta "node_modules/"?

"package.json" registra las dependencias y scripts del proyecto[cite: 1]. "node_modules/" almacena los archivos físicos de las dependencias instaladas (los cuales no deben modificarse manualmente)[cite: 1].

--- SECCIÓN 5: COMPONENTES Y SINTAXIS JSX ---

15. Desde el punto de vista del código JavaScript, ¿qué es exactamente un componente de React?

Es simplemente una función de JavaScript que retorna elementos de interfaz de usuario[cite: 1].

16. ¿Cuál es la "Regla de Oro" respecto a la nomenclatura (mayúsculas/minúsculas) al nombrar un componente y por qué es necesaria?

Deben iniciar siempre con mayúscula utilizando la convención PascalCase[cite: 1]. Es necesaria para que React diferencie los componentes propios de las etiquetas HTML estándar[cite: 1].

17. ¿Cuáles son las 4 características principales de la sintaxis JSX descritas en el manual?

1. Cohesión entre Lógica + Markup en el mismo archivo[cite: 1].
2. Uso de Fragmentos (los componentes deben retornar un único elemento padre)[cite: 1].
3. Uso de `className` en lugar de `class`[cite: 1].
4. Uso de llaves `{}` para evaluar variables o funciones de JS dentro de la UI[cite: 1].

18. ¿Por qué en JSX se utiliza el atributo "className" en lugar de "class"?

Porque `class` es una palabra reservada dentro del lenguaje JavaScript[cite: 1].

19. ¿Qué sucede "debajo del capó" con el código JSX antes de ser interpretado por el navegador y qué compiladores intervienen?

El navegador no entiende JSX, por lo que compiladores como Babel o SWC transforman la sintaxis en llamadas a la función `React.createElement(...)`[cite: 1].

--- SECCIÓN 6: FLUJO DE DATOS: PROPS Y COMPOSICIÓN ---

20. ¿Qué son las "Props" en React y cómo permiten la comunicación entre un componente Padre y un componente Hijo?

Son las propiedades o argumentos que un componente Padre le pasa a un componente Hijo para transferirle datos o configuraciones[cite: 1].

21. ¿En qué consiste la propiedad especial "children" y qué capacidades otorga para la composición de la interfaz?

Es una prop especial que permite anidar componentes o contenido HTML dentro de otros componentes, facilitando la creación de estructuras complejas y reutilizables[cite: 1].

--- SECCIÓN 7: EL PROBLEMA DEL DOM REAL VS. LA SOLUCIÓN DEL VIRTUAL DOM ---

22. ¿Cuáles son las principales limitaciones y deficiencias de rendimiento asociadas al DOM Real del navegador?

El repintado del DOM Real es la operación más costosa para el navegador[cite: 1]. Buscar nodos y actualizarlos manualmente con JS es tedioso, propenso a errores y poco escalable[cite: 1].

23. ¿Qué es el "Virtual DOM" y cómo funciona el proceso de 3 pasos (Copia, Diffing, Actualización Quirúrgica)?

Es una copia ligera del DOM Real guardada en memoria[cite: 1]. Funciona así:

1. Copia en Memoria: Mantiene una representación del DOM[cite: 1].
2. Comparación (Diffing): Compara la copia nueva con la anterior cuando cambia el estado[cite: 1].
3. Actualización Quirúrgica: Aplica en el DOM Real únicamente los cambios exactos, sin volver a renderizar toda la página[cite: 1].

24. ¿En qué consiste el algoritmo/proceso de "Diffing" o reconciliación?

Es el proceso en el que React compara el árbol del Virtual DOM nuevo con el anterior para calcular las diferencias mínimas entre ambos[cite: 1].

--- SECCIÓN 8: SEPARACIÓN DE RESPONSABILIDADES Y EL CICLO DE REACT ---

25. ¿A qué se refiere el principio de que "React NO pinta la pantalla"? ¿Qué rol juegan "ReactDOM" y "React Native"?

Significa que React solo calcula el Virtual DOM y gestiona el estado[cite: 1]. Luego delega el trabajo de renderizado a "React DOM" para pintar HTML en la web, o a "React Native" para generar componentes nativos en móviles[cite: 1].

26. Explica detalladamente las 5 etapas del ciclo completo de React (Interacción, Cambio de Estado, Re-renderizado, Reconciliación y Actualización).

1. Interacción: El usuario realiza una acción en la UI[cite: 1].
2. Cambio de Estado: Un Hook (como useState) procesa la actualización[cite: 1].
3. Re-renderizado: React genera un nuevo Virtual DOM con los datos actualizados[cite: 1].
4. Reconciliación: Compara mediante Diffing el Virtual DOM nuevo contra el antiguo[cite: 1].
5. Actualización: ReactDOM aplica las diferencias mínimas en el DOM Real del navegador[cite: 1].

27. ¿Qué Hook se encarga de detectar actualizaciones para desencadenar el ciclo de re-renderizado?

El Hook `useState`[cite: 1].